

ShopSmart Step by Step Laboratory Guide

Three hours | JavaScript and React | Student handout

In this laboratory you will run the instructor supplied ShopSmart starter, inspect its current behavior, then make three small improvements. The application already has working cart controls, stock validation, an Express API, and a PostgreSQL catalog. Follow the steps in order and keep a copy of your original files.

What you will produce

- Product search that checks both product names and descriptions while preserving the category filter.
- A product card that shows “Only N left” for stock levels from 1 to 8.
- One reusable cart item count function used by the navigation and order summary.

Preparation and setup 0 to 30 minutes

Step 1 Extract and open the correct folder

Extract ShopSmart-Starter.zip. In VS Code select File > Open Folder and choose the extracted ShopSmart-Starter directory. The top level must contain package.json, frontend, backend, docs, and README.md. Open Terminal > New Terminal. All commands below run in that top-level folder unless stated otherwise.

Why this matters in ShopSmart: The top-level package.json defines commands for both the frontend and backend. Running them from only frontend or backend can produce a different result.

Step 2 Check Node.js and npm

```
node --version  
npm --version
```

Both commands should print version numbers. The project requires Node.js 20.19 or newer. If either command is missing, follow the installation section in README.md and reopen VS Code before continuing.

Why this matters in ShopSmart: Node.js runs the development tools and the backend API. npm installs the packages listed in the project.

Step 3 Complete first-time setup only if this is a fresh machine or fresh extraction

If this extracted folder is already configured and runs, skip this step. For a fresh extraction, create the environment files and install dependencies. In a PowerShell terminal at the project root, run each command in order:

```
Copy-Item backend\.env.example backend\.env  
Copy-Item frontend\.env.example frontend\.env  
npm install  
npm run db:generate
```

On macOS or Linux use `cp backend/.env.example backend/.env` and `cp frontend/.env.example frontend/.env` instead of the two Copy-Item commands. Then use the same npm commands. Open backend/.env and match its DATABASE_URL user, password, port, and database name to your local PostgreSQL setup. Keep both .env files private.

If you have not created the ShopSmart database before, follow README.md to create the database user and database. With PostgreSQL running, create the tables and insert the six example products:

```
npm run db:migrate
npm run db:seed
```

Expect “Seeded 6 ShopSmart products.” If your ShopSmart database already contains classroom work, do not run db:seed: it deletes orders and restores sample products. Do not create a second database or overwrite existing .env files. Ask the instructor if you cannot tell whether the database already contains work.

Why this matters in ShopSmart: The backend needs its database connection, and Prisma needs a generated client and tables. Seeding puts the example product objects in PostgreSQL so the frontend can retrieve them through the API.

Step 4 Start the application

```
npm run dev
```

Keep this terminal open. Visit <http://localhost:3000/api/health> and check for `{"status":"ok"}`. Then open <http://localhost:5173> and look for six product cards. If products are missing, check that PostgreSQL is running and read the first backend error in the terminal. Use the README.md troubleshooting instructions.

Why this matters in ShopSmart: The health page confirms the API process is available. Seeing product cards also confirms that the frontend fetched catalog data from the API and database.

Part A Explore the starter 30 to 45 minutes

Step 5 Trace the data without editing

Open `frontend/src/pages/ProductsPage.jsx`. Find the products state, the call to `api.getProducts()`, `visibleProducts`, and the map that creates `ProductCard` components. Next open `frontend/src/components/ProductCard.jsx` and observe `product.name`, `product.description`, `product.stock`, and the Add to cart `onClick` handler. Finally open `frontend/src/context/CartContext.jsx` and `frontend/src/utls/cart.js`.

What to find	File and expression to identify
Product array	<code>ProductsPage.jsx: products state</code>
Filter	<code>ProductsPage.jsx: visibleProducts</code>
Click event	<code>ProductCard.jsx: onClick</code>
Cart state	<code>CartContext.jsx: cart state</code>
Total calculation	<code>cart.js: calculateCartTotal</code>

Why this matters in ShopSmart: The API fills products state, `ProductsPage` filters the array, and `ProductCard` renders each object. `CartContext` holds cart state and calls reusable calculation functions. This map tells you where an edit belongs.

Step 6 Record baseline behavior

Search for keyboard, clear search, add the keyboard twice, and open Cart. Confirm its quantity is 2. Change quantity to 1 and remove it. The current application should perform these actions before your edits. Write down any pre-existing error and show it to your instructor.

Why this matters in ShopSmart: This is your comparison point. Later you can tell whether your changes improved ShopSmart without breaking its existing cart behavior.

Part B Improve product search 45 to 85 minutes

Step 7 Locate the search expression

In `frontend/src/pages/ProductsPage.jsx` find this existing code inside `visibleProducts`:

```
const matchesSearch = product.name.toLowerCase().includes(search.trim().toLowerCase());
```

Why this matters in ShopSmart: The current code checks only names. A buyer who remembers a feature from a description may not find a suitable product.

Step 8 Use one normalized query and check two fields

Add the query variable immediately before const visibleProducts. Replace only the matchesSearch expression as shown. Keep matchesCategory and return unchanged.

```
const query = search.trim().toLowerCase();

const visibleProducts = products.filter((product) => {
  const matchesSearch =
    product.name.toLowerCase().includes(query) ||
    product.description.toLowerCase().includes(query);
  const matchesCategory = category === 'All' || product.category === category;
  return matchesSearch && matchesCategory;
});
```

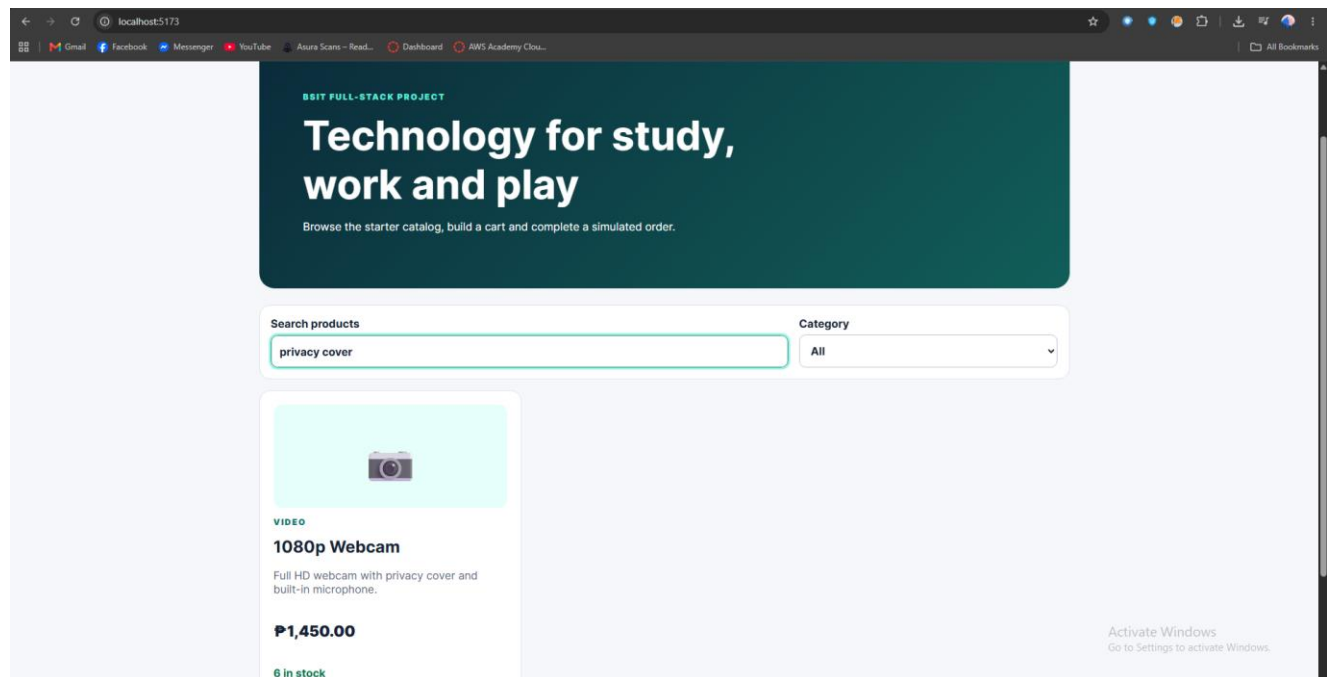
The || operator accepts a match in either text field. The final && still requires the selected category to match. Save the file. Vite should refresh the browser automatically.

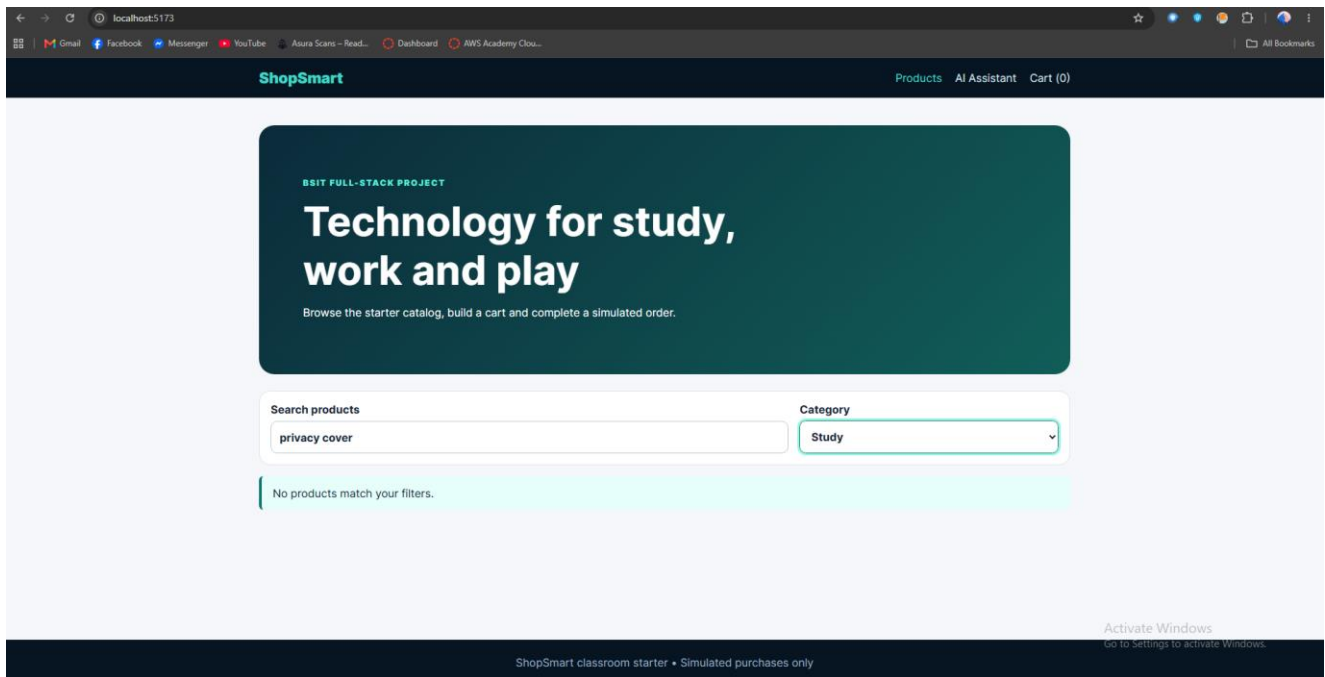
Why this matters in ShopSmart: ShopSmart will now find products by features written in descriptions, while the category selector still narrows the displayed results.

Step 9 Test the search cases

Action	Expected result
Search privacy cover with All categories	1080p Webcam appears
Search keyboard	Compact Mechanical Keyboard appears
Search privacy cover and select Study	No products match your filters
Clear the search while Study is selected	Study products appear

Record one screenshot of the privacy cover result. If the result does not change, check whether query was placed before visibleProducts and whether both fields use toLowerCase().





Why this matters in ShopSmart: These four cases check the new feature and guard against accidentally breaking name search or category filtering.

Part C Improve stock labels 85 to 120 minutes

Step 10 Inspect the stock and button logic

Open `frontend/src/components/ProductCard.jsx`. Find the stock paragraph and the button condition `disabled={product.stock < 1}`. Keep the button condition and the paragraph `className` unchanged.

Why this matters in ShopSmart: The stock paragraph informs the buyer. The disabled condition separately prevents an out-of-stock item from being added, so preserve both responsibilities.

Step 11 Compute a label before return

Inside `ProductCard`, after the opening function line and before return, add:

```
let stockLabel;
if (product.stock < 1) {
  stockLabel = 'Out of stock';
} else if (product.stock <= 8) {
  stockLabel = `Only ${product.stock} left`;
} else {
  stockLabel = `${product.stock} in stock`;
}
```

Why this matters in ShopSmart: A conditional turns a numeric database stock value into a message that tells buyers when few units remain.

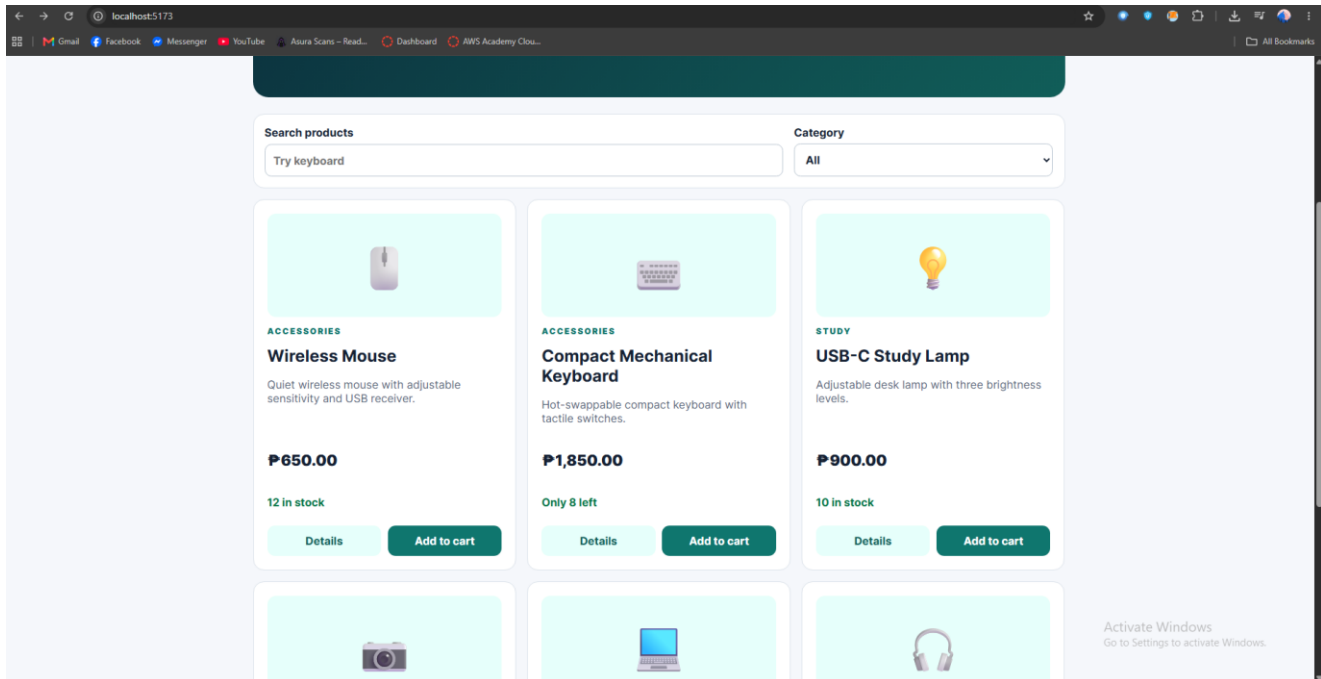
Step 12 Display the new label

In the existing stock paragraph, replace only the contents between its opening and closing `p` tags with `{stockLabel}`. The complete paragraph should read:

```
<p className={product.stock > 0 ? 'stock' : 'stock unavailable'}>
  {stockLabel}
</p>
```

Save and reload. The seed catalog has stock values 6, 7, 8, 9, 10, and 12: you can observe both positive-stock labels without changing the database. For stock 0, explain the first if branch and show that the original disabled condition still handles zero. Do not reset database records simply to generate a screenshot.

- The first branch of the code checks if the product's stock is less than one, and if it is, it sets stockLabel to "Out of stock." The code then displays the message indirectly with the existing stock being checked with the condition disabled={product.stock<1} disabling the Add to cart button. As for positive-stock seeded products, the stock displaying 6, 7, and 8 shows the message "Only N left," while in case of 9, 10, and 12 it shows "N in stock." I checked the positive-stock labels without modifying the database.



```
export default function ProductCard({ product, onAddToCart }) {
  let stockLabel;
  if (product.stock < 1) {
    stockLabel = 'Out of stock';
  } else if (product.stock <= 8) {
    stockLabel = `Only ${product.stock} left`;
  } else {
    stockLabel = `${product.stock} in stock`;
  }

  return (
    <article className="product-card">
      <div className="product-icon" aria-hidden="true">{product.emoji}</div>
      <p className="eyebrow">{product.category}</p>
      <h2>{product.name}</h2>
      <p className="product-description">{product.description}</p>
      <p className="price">₱{Number(product.price).toLocaleString('en-PH', { minimumFractionDigits: 2 })}</p>
      <p className={product.stock > 0 ? 'stock' : 'stock unavailable'}> {stockLabel}</p>
      <div className="card-actions">
        <Link className="button secondary" to={`/products/${product.id}`}>Details</Link>
        <button disabled={product.stock < 1} onClick={() => onAddToCart(product)}>Add to cart</button>
      </div>
    </article>
  );
}
```

Why this matters in ShopSmart: Changing only the displayed text preserves ShopSmart's existing add-to-cart and stock checks. The two positive-stock cases can be verified with seeded products; the zero case is covered by the code condition.

Part D Reuse the cart count 120 to 155 minutes

Step 13 Write the calculation once

Open frontend/src/Utils/cart.js. Add this exported function after calculateCartTotal. This counts individual units, so quantities 2 and 3 add to 5.

```
export function calculateCartItemCount(cart) {  
  return cart.reduce((count, item) => count + item.quantity, 0);  
}
```

Why this matters in ShopSmart: The number of units differs from the number of distinct products and from the peso total. An empty cart correctly starts at 0.

Step 14 Use the function in cart context

Open frontend/src/context/CartContext.jsx. Replace the existing import from ../utils/cart.js with this line. In the value object, replace the inline itemCount reduce expression as shown:

```
import {  
  addProductToCart, calculateCartItemCount,  
  calculateCartTotal, updateCartQuantity  
} from '../utils/cart.js';
```

```
itemCount: calculateCartItemCount(cart),
```

Keep total: calculateCartTotal(cart), unchanged. The navigation already reads itemCount from this context in frontend/src/App.jsx.

Why this matters in ShopSmart: CartContext now exposes one trusted item count to any component that calls useCart(), including the navigation bar.

Step 15 Use the same count in the order summary

Open frontend/src/pages/CartPage.jsx. Add itemCount to the names destructured from useCart(). Find the order summary row labeled Items and replace its inline cart.reduce(...) expression with {itemCount}. Do not change the total row.

```
const { cart, total, itemCount, cartMessage, setQuantity, removeFromCart } = useCart();
```

```
<p><span>Items</span><strong>{itemCount}</strong></p>
```

Replace the existing Items row with the p element shown above. Keep the Total row and the other cart markup unchanged.

Why this matters in ShopSmart: The order summary and navigation now show the same quantity because both read the count that CartContext calculated.

Step 16 Check the behavior in the browser

Add one product twice and another product once. The top navigation must say Cart (3) and the Items row must say 3. Change a quantity to 3: both numbers must change to 4. Remove the other product: both numbers must change to 3. The monetary total should still reflect price × quantity.

Why this matters in ShopSmart: These actions exercise adding, changing, and removing items. They show whether the shared count updates whenever cart state changes, while the price calculation stays independent.

Part E Verify and submit 155 to 180 minutes

Step 17 Run the existing frontend tests

Open a second terminal at the ShopSmart-Starter root while the first terminal continues running npm run dev. Run one command at a time:

```
npm run test --workspace frontend
```

This command runs the existing frontend Vitest suite. It should exit without errors, but the original tests do not cover your new count function. Add that case in the next step.

Why this matters in ShopSmart: Existing tests check that cart totals and stock rules still behave as they did before the lab.

Step 18 Add one targeted test for the new function

In frontend/src/utils/cart.test.js, replace the existing import from ./cart.js with the following import. Add the test inside the describe block, after any existing test:

```
import {
  addProductToCart, calculateCartItemCount,
  calculateCartTotal, updateCartQuantity
} from './cart.js';

it('counts all cart units and handles an empty cart', () => {
  expect(calculateCartItemCount([{ quantity: 2 }, { quantity: 3 }])).toBe(5);
  expect(calculateCartItemCount([])).toBe(0);
});
```

From the same second terminal, run the focused test and then the full project check:

```
npm run test --workspace frontend
npm run check
```

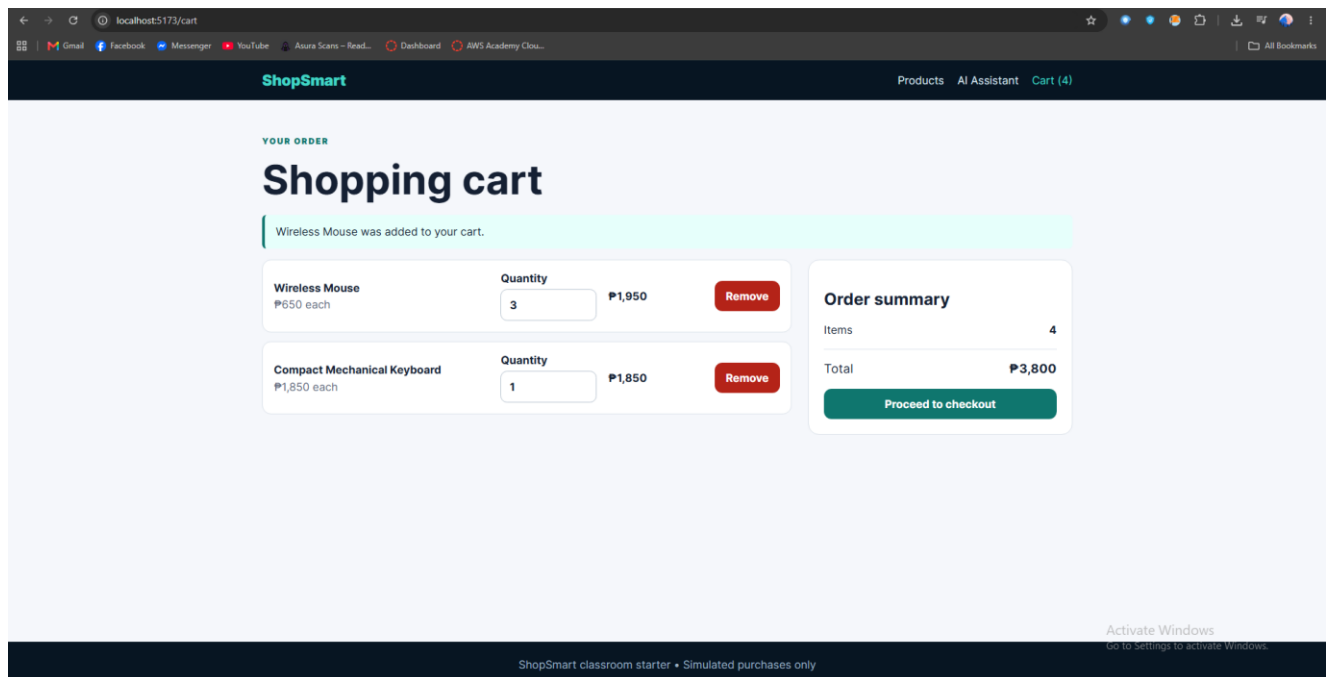
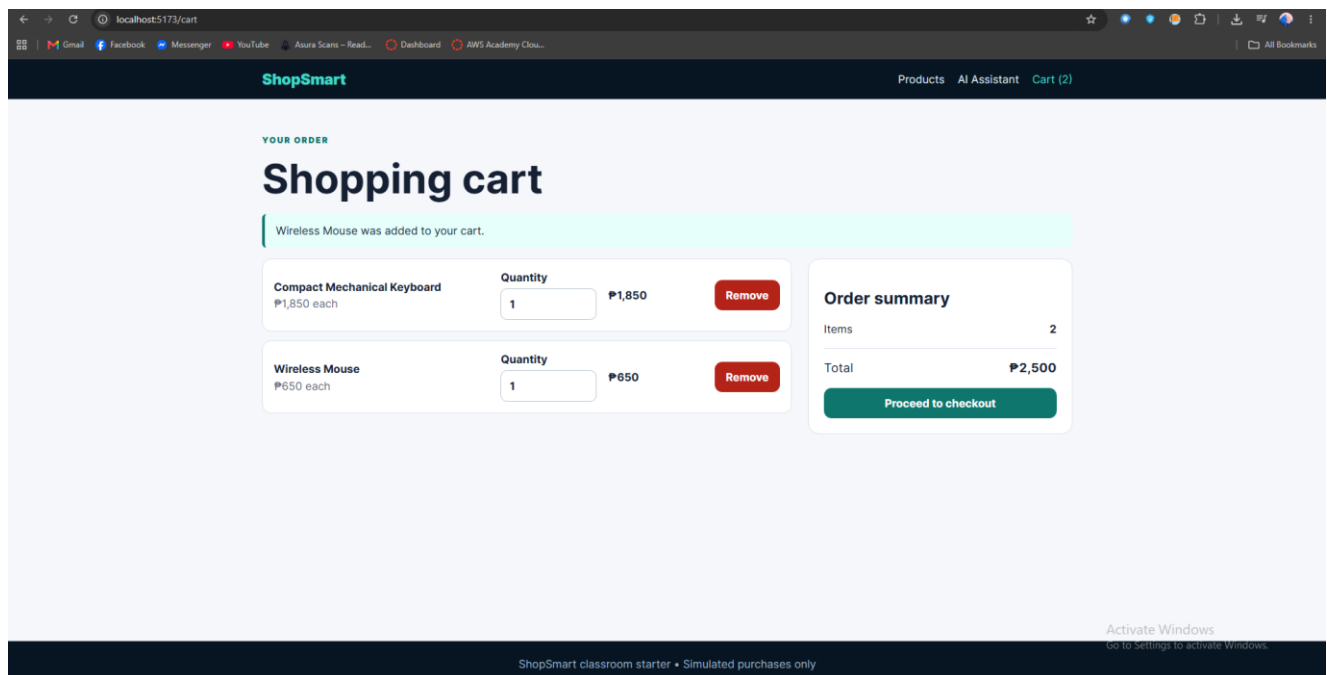
The full check runs both linters, all tests, and a frontend build. Both commands should exit without errors. If a check fails, read its first relevant error, find the named file and line, correct the issue, and rerun the same command.

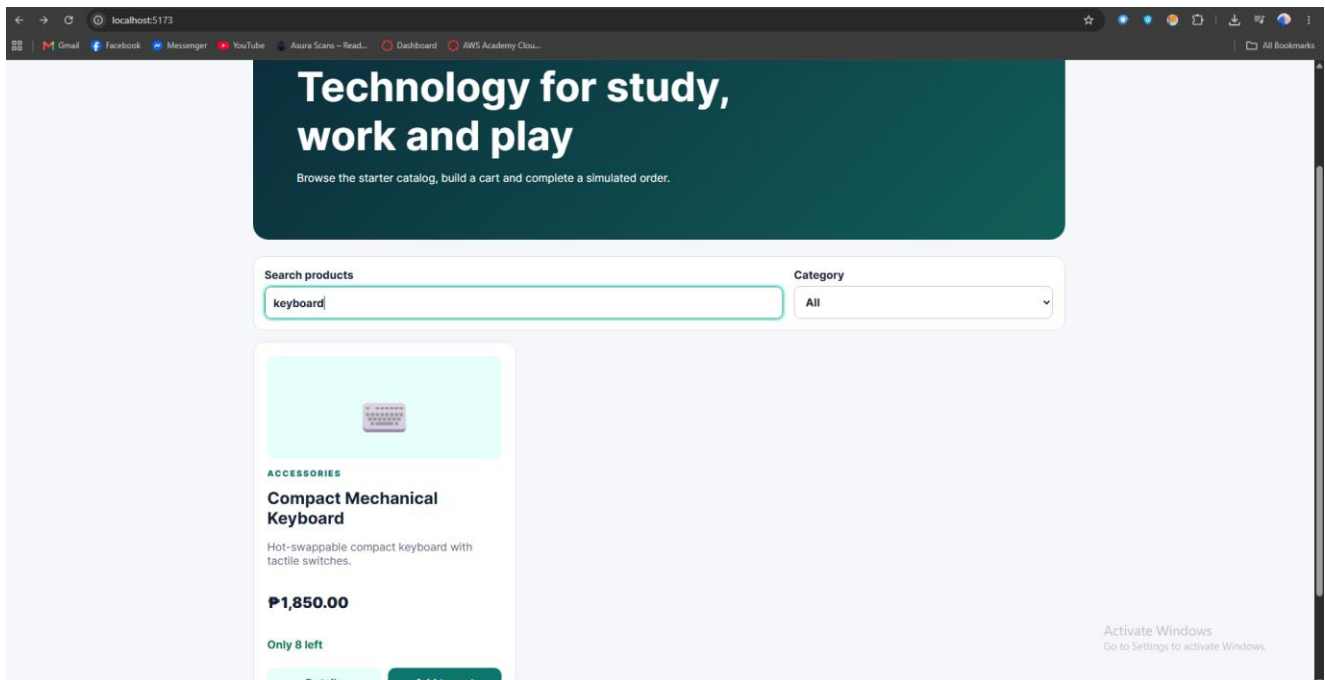
Why this matters in ShopSmart: The new test checks the calculation you added. The full check also catches syntax, lint, and build problems elsewhere in ShopSmart before submission.

Step 19 Submit your work

- Submit the edited project source or a repository link. Exclude node_modules, backend/.env, and frontend/.env.
- Include screenshots of the description search, a low stock label, and the cart item count after increasing quantity.
- Include the final npm run check result and three to five sentences explaining the search condition, stock branches, and reduce function.

Why this matters in ShopSmart: Source code and screenshots show the visible result; the command output and explanation show that you checked the application and understand your changes.





```
Problems Output Debug Console Terminal Ports
PS C:\Users\Username\Documents\BSIT\ShopSmart-Starter> npm run check

> shoppmart-starter@1.0.0 check
> npm run lint --workspace frontend && npm run lint --workspace backend && npm run test && npm run build

> shoppmart-frontend@1.0.0 lint
> eslint .

> shoppmart-backend@1.0.0 lint
> eslint .

> shoppmart-starter@1.0.0 test
> npm run test --workspace frontend && npm run test --workspace backend

> shoppmart-frontend@1.0.0 test
> vitest run

RUN v3.2.7 C:/Users/Username/Documents/BSIT/ShopSmart-Starter/frontend
✓ src/utlils/cart.test.js (4 tests) 3ms
  ✓ cart utilities > calculates the cart total 1ms
  ✓ cart utilities > adds a new product 0ms
  ✓ cart utilities > prevents a quantity above stock 0ms
  ✓ cart utilities > counts all cart units and handles an empty cart 0ms
Test Files 1 passed (1)
Tests 4 passed (4)
Start at 13:39:30
Duration 1.05s (transform 38ms, setup 0ms, collect 32ms, tests 3ms, environment 0ms, prepare 438ms)

> shoppmart-backend@1.0.0 test
> node --test

✓ parseBudget extracts a peso budget (1.2644ms)
✓ rankProducts respects the budget (2.3908ms)
✓ assistant response contains traceable sources (0.2689ms)
✓ a valid classroom order passes validation (2.8247ms)
✓ duplicate products fail order validation (0.3285ms)
✓ an oversized assistant question fails validation (0.473ms)
# tests 6
# suites 0
# pass 6
# fail 0
# cancelled 0
# skipped 0
# todo 0
# duration_ms 235.3413
```

```
> shoppmart-starter@1.0.0 build
> npm run build --workspace frontend

shoppmart-frontend@1.0.0 build
> vite build

vite v7.3.6 building client environment for production...
../node_modules/react-router/dist/development/index.mjs (11:0): Module level directives cause errors when bundled, "use client" in "../node_modules/react-router/dist/development/index.mjs" was ignored.
../node_modules/react-router/dist/development/dom-export.mjs (1:0): Module level directives cause errors when bundled, "use client" in "../node_modules/react-router/dist/development/dom-export.mjs" was ignored.
✓ 47 modules transformed.
dist/index.html 0.49 kB | gzip: 0.30 kB
dist/assets/index-C0wcbWnj.css 4.30 kB | gzip: 1.48 kB
dist/assets/index-CrK0YYMO.js 272.25 kB | gzip: 86.06 kB
✓ built in 1.26s
PS C:\Users\Username\Documents\BSIT\ShopSmart-Starter>
```

- The search converts the query to lowercase and looks where it occurs in the product name or description as well as checks that the selected category matches. The stock label shows “Out of stock” when less than 1 is in stock, “Only N left” for between 1 and 8, and “N in stock” when more than 8 stocks are available. The Add to Cart button is disabled when less than 1 is in stock, moreover, the cart logic does not allow adding more items than in the stock. The cart item count uses reduce to calculate the quantity of each item in the cart, that’s why the amount of items increases when one product number increases.

Assessment 100 points

Setup and inspection: 15 points. Description search with category filter: 25. Stock labels and existing button behavior: 20. Reusable item count and targeted test: 25. Browser verification, explanation, and clean submission: 15.

If the project stops after the session, return to the first terminal and press Ctrl+C. On the next class day, start PostgreSQL and run npm run dev again from the project root. There is no need to reseed the database.